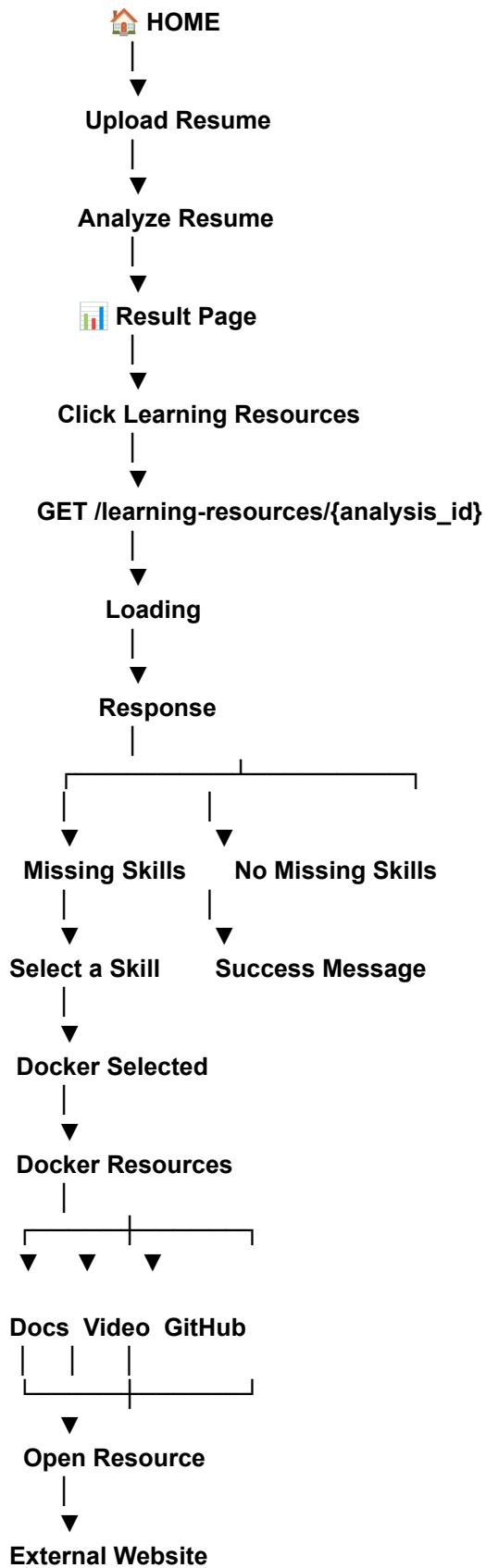
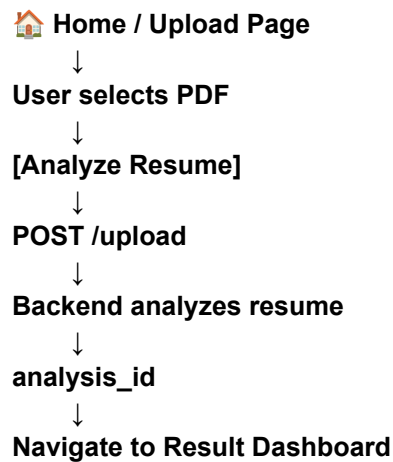




# 1. User uploads resume

This is your existing V1 flow.



Your existing upload page does not need to change significantly.

---

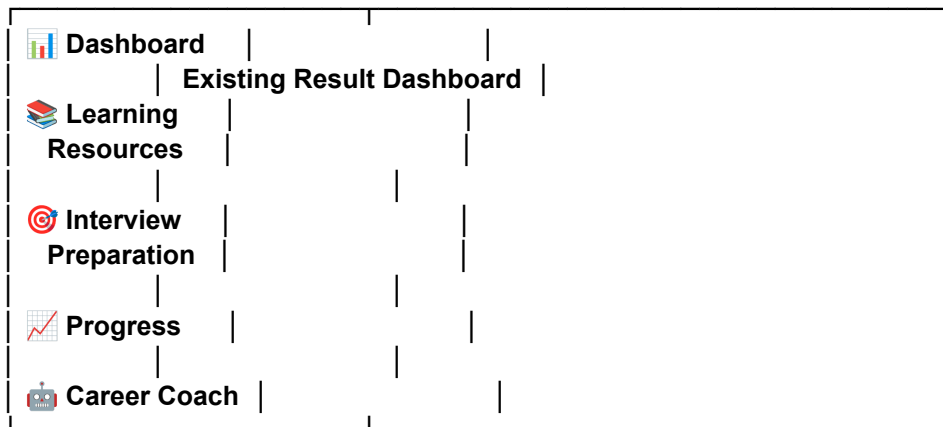
# 2. Result Dashboard

User reaches your existing Result Page.

📊 Resume Analysis Dashboard

ATS Score  
Professional Summary  
Technical Skills  
Soft Skills  
Missing Skills  
Suggestions  
Learning Roadmap  
Suggested Projects  
Estimated Timeline

Now V2 adds the sidebar navigation.



The user clicks:

 **Learning Resources**

---

### 3. Learning Resources Page opens

Frontend calls your existing backend API:

**GET /learning-resources/{analysis\_id}**

Flow:

**Learning Resources Page**



**Get analysis\_id**



**GET /learning-resources/{analysis\_id}**



**Backend**



**Analysis**



**Missing Skills**



**Learning Resources table**



**Response**



**Frontend**

---

## 4. Loading state

While the API is working:

 Learning Resources

Resources recommended based on your resume analysis.

Loading resources...



We should use a skeleton/loading UI rather than showing a blank screen.

---

## 5. Display Missing Skills

After successful response:

 Learning Resources

Your Recommended Skills

[ Docker ]  
[ AWS ]  
[ PyTorch ]  
[ Kubernetes ]  
[ Advanced SQL ]

These are not hardcoded.

They come from the analysis/backend response.

---

## 6. Select a skill

Suppose the user clicks:

[ Docker ]

The UI changes the selected skill:

Selected Skill: Docker

Then show:

Resources for Docker

The page doesn't navigate to another page.

It's still the same Learning Resources page.

Learning Resources



Select Docker



Display Docker resources

---

## 7. Display Resource Cards

For Docker:

Resources for Docker

|                                                                                                                   |  |
|-------------------------------------------------------------------------------------------------------------------|--|
|  Docker Official Documentation |  |
| DOCUMENTATION                                                                                                     |  |
| [Open Resource →]                                                                                                 |  |

► Docker Tutorial for Beginners

VIDEO

[Watch Video →]

 Docker Documentation Repository

GITHUB

[Open GitHub →]

The frontend receives:

skill  
resource\_type  
title  
url

and renders the cards dynamically.

---

## 8. Resource button

When the user clicks:

[Open Resource]

the frontend opens the stored:

url

in a new browser tab.

So:

Frontend



Open Resource



External website

The user doesn't leave your Career Coach application unnecessarily.

---

## 9. Resource types

Your backend already supports different resource types, so the frontend can visually distinguish them: DOCUMENTATION

VIDEO  
GITHUB  
COURSE

ARTICLE  
OTHER

For example:

 DOCUMENTATION  
 VIDEO  
 GITHUB  
 COURSE  
 ARTICLE  
 OTHER

The exact icon isn't important right now; consistent visual treatment is.

---

## 10. No resources available

Suppose analysis identifies:

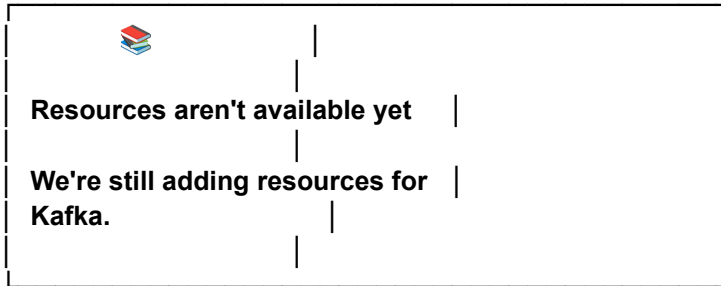
Kafka

but your database doesn't have Kafka resources.

Your backend already returns something like:

Learning resources for Kafka are not available yet.

Frontend should show a professional empty state:



Don't show an empty white screen.

---

## 11. No missing skills

If the resume has no missing skills:

 Great!

No major missing skills were identified  
from your resume analysis.

Keep improving your existing skills.

Again, this is a proper empty/success state.

---

## 12. API error

If the backend fails:

 Unable to load learning resources.

Something went wrong while loading



your personalized resources.

[Retry]

Clicking Retry calls:

GET /learning-resources/{analysis\_id}

again.

---

## 13. User changes skill

Suppose:

Docker selected

Then user clicks:

AWS

The UI becomes:

Selected Skill: AWS

Resources for AWS



AWS Documentation

AWS Course

AWS Video

...

No page reload is necessary.

---

## 14. User moves to another feature

Sidebar remains available:



Dashboard



Learning Resources ← current



Interview Preparation



Learning Progress



Career Coach

User can directly move to:

**Learning Resources**



**Interview Preparation**

or:

**Learning Resources**



**Learning Progress**

or:

**Learning Resources**

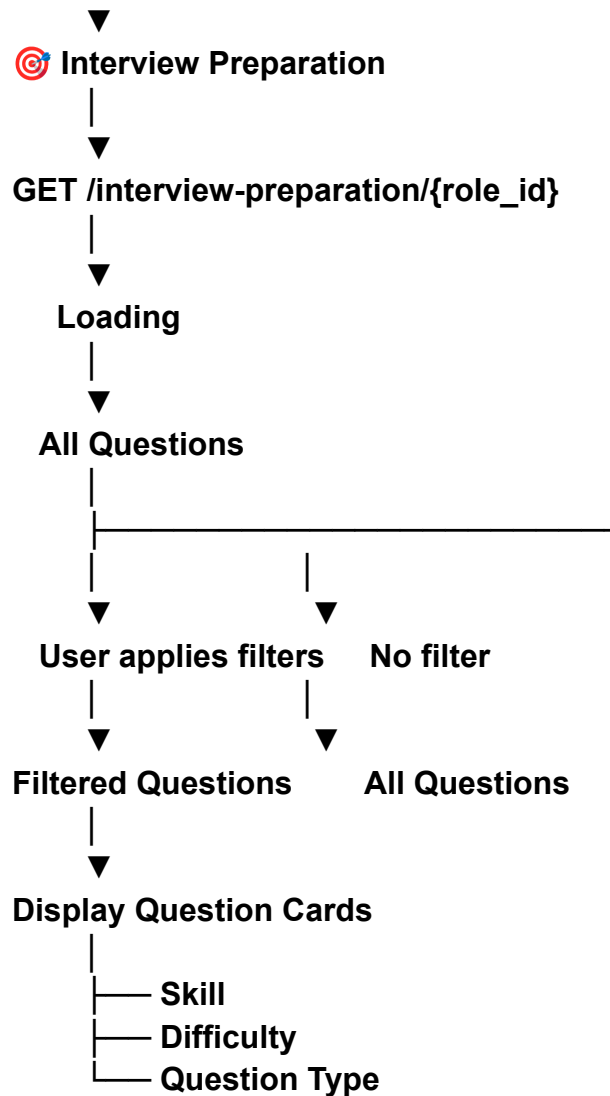


**Career Coach'**



**Result Dashboard**





## Interview Preparation — V2 Design

The purpose of this page is:

Help the user practice interview questions based on their career role and skills.

Your backend already supports:

role\_id

skill

difficulty

question\_type

So the frontend should make those filters easy to use.

---

## 1 User opens Interview Preparation

From the sidebar:



Dashboard



Learning Resources



Interview Preparation ← User clicks



Learning Progress



Career Coach

Page:



Interview Preparation

Prepare for interviews based on your role and skills.

---

## 2 Show the user's role

The backend already knows the role through `role_id`.

For example:

Role: Backend Developer

So the page could show:

 Interview Preparation

Your Role

|                   |  |
|-------------------|--|
| Backend Developer |  |
|-------------------|--|

The role should not be manually typed by the user.

---

### 3 Filter section

Your backend supports three filters:

Skill

Difficulty

Question Type

So the frontend should have:

Skill

[ All Skills ▼ ]

Difficulty

[ All Difficulties ▼ ]

Question Type

[ All Types ▼ ]

For example:

| Skill       | Difficulty       | Question Type |
|-------------|------------------|---------------|
| [FastAPI ▼] | [Intermediate ▼] | [Coding ▼]    |

And:

[ Apply Filters ]

#### 4 Important: Default state

When the user first opens the page, I recommend:

Skill: All

Difficulty: All

Question Type: All

So the user immediately sees all available interview questions for their role.

This matches your original API design:

GET /interview-preparation/{role\_id}

Then filters can narrow the result.

#### 5 Questions display

Example:

Role: Backend Developer

12 Questions Found

Then cards:

|                                          |   |              |          |
|------------------------------------------|---|--------------|----------|
| Q1                                       |   |              |          |
| What is dependency injection in FastAPI? |   |              |          |
| FastAPI                                  | • | Intermediate | • Theory |

Another:

|                                 |   |              |          |
|---------------------------------|---|--------------|----------|
| Q2                              |   |              |          |
| Build a CRUD API using FastAPI. |   |              |          |
| FastAPI                         | • | Intermediate | • Coding |

The backend already provides:

skill

question

question\_type

difficulty

So the frontend renders these dynamically.

---

## ⑥ Filter interaction

Suppose the user chooses:

Skill: FastAPI

Difficulty: Intermediate

Question Type: Coding

Then:

[ Apply Filters ]



GET /interview-preparation/{role\_id}

?skill=FastAPI

&difficulty=Intermediate

&question\_type=Coding



Backend



Filtered Questions



Frontend

Only matching questions appear.

---



## 7 Reset Filters

We should also provide:

[ Apply Filters ] [ Reset ]

If the user clicks Reset:

Skill → All

Difficulty → All

Question Type → All

Then fetch all questions again.

This is much better UX than forcing the user to manually undo each filter.

---

## 8 Loading state

When questions are being fetched:

 Interview Preparation

Loading interview questions...



Use skeleton cards or a clean loading indicator.

---

## 9 No questions found

Suppose the user selects:

Skill: FastAPI

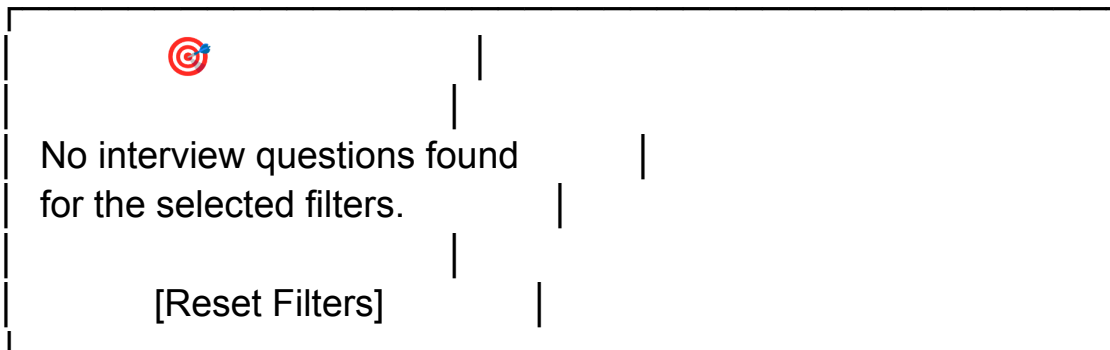
Difficulty: Advanced

Question Type: Coding

and no matching questions exist.

Don't show a blank page.

Show:



## 10 API error

If the backend fails:

⚠ Unable to load interview questions.

Something went wrong.

[Retry]

Retry calls the same API again.

---

### Question interaction

For V1, I recommend keeping questions simple.

The user can:

Read question



Practice answer

We don't need to build a full online coding editor or answer evaluation system yet.

That can become a future V2/V3 feature.

For now:

Question

Skill

Difficulty

Question Type

is enough.



Dashboard



Learning Progress



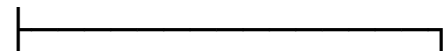
GET /progress-tracking/{resume\_id}



Loading



Response



Progress Exists    No Progress



Show Dashboard    0% State

|

|— Progress %

|— Completed Skills

|— Current Stage

|



[Update Progress]



Update Form / Modal



PUT /progress-tracking/{resume\_id}



Updated Database

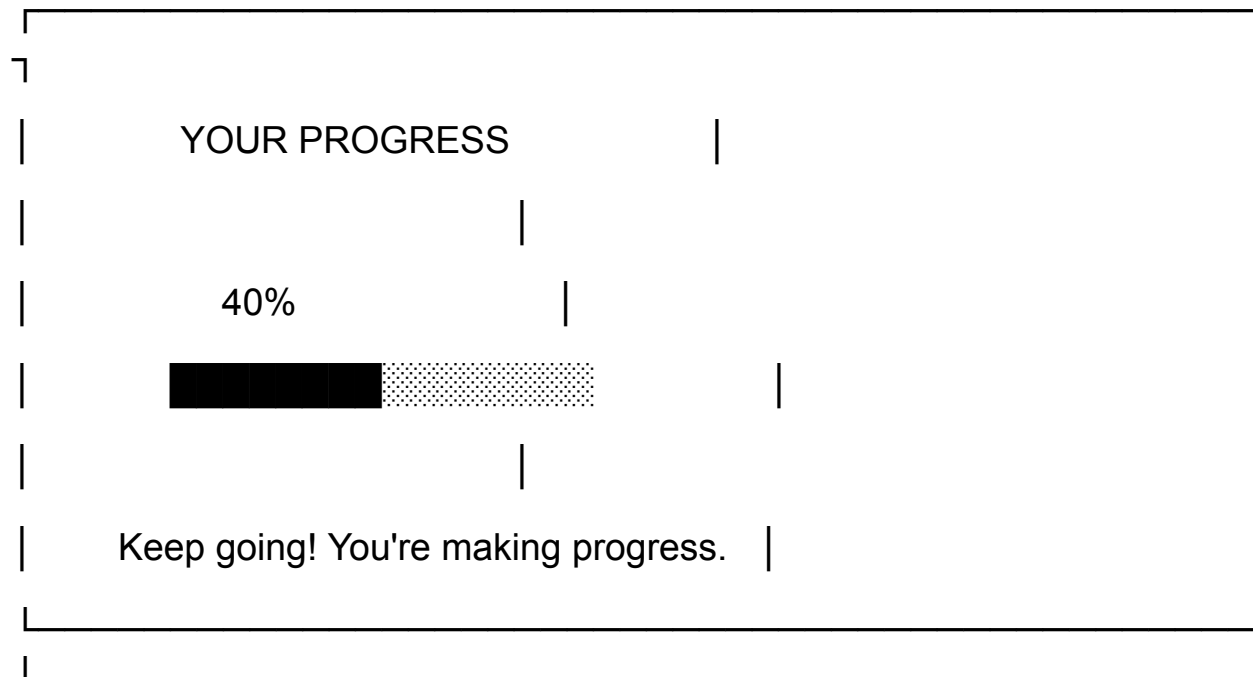


Refresh UI

## 2 Progress overview

The most important information should be immediately visible.

Example:



Your backend field:

progress

is the percentage from **0–100**.

## 3 Current Learning Stage

Show:

Current Learning Stage

Docker

Your backend field:

current\_learning\_stage

So the user immediately knows:

**What am I currently learning?**

---

## **4 Completed Skills**

Show:

Completed Skills

✓ Python

✓ FastAPI

✓ PostgreSQL

Your backend field:

completed\_skill

Depending on the format you've stored, the frontend can display the completed skills cleanly.

For example:

\_\_\_\_\_

| ☒ Python |  
|\_\_\_\_\_|

|\_\_\_\_\_|  
| ☒ FastAPI |  
|\_\_\_\_\_|

|\_\_\_\_\_|  
| ☒ Docker |  
|\_\_\_\_\_|

---

## 5 Learning Journey

For a more professional V2 UI, I would add a visual roadmap.

Example:

Learning Journey

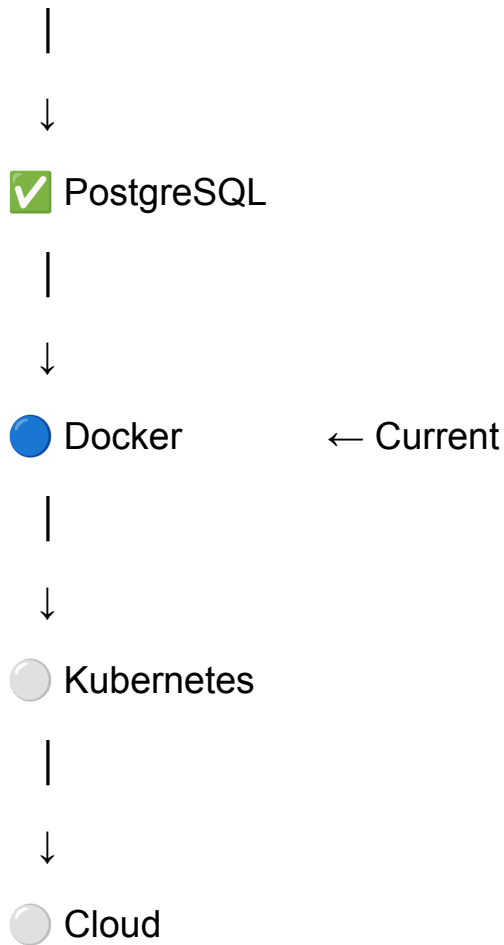
☒ Python

|

↓

☒ FastAPI





### Important

This visual roadmap should be based on the user's learning data/roadmap.

We shouldn't invent a completely separate learning system in the frontend.

The frontend is **visualizing the information your backend already provides.**

---

## 6 Update Progress

The user needs a clear action:

[  Update Progress ]

When clicked, open a modal/form:

|                          |  |
|--------------------------|--|
| Update Learning Progress |  |
|                          |  |
| Progress                 |  |
| [ 40 ] %                 |  |
|                          |  |
| Completed Skill          |  |
| [ FastAPI, PostgreSQL ]  |  |
|                          |  |
| Current Learning Stage   |  |
| [ Docker ]               |  |
|                          |  |
| [Cancel] [Save]          |  |

Then:

Save



PUT /progress-tracking/{resume\_id}



Backend updates DB



Frontend refreshes progress

---

## **7 Progress validation**

Frontend should prevent obvious invalid values.

For example:

Progress:

0 ————— 100

Don't allow:

-10

150

abc

The backend should still validate it too.

Frontend validation is for UX; backend validation is for security/data integrity.

---

## **8 Reset Progress**

You already have:

PUT /progress-tracking/{resume\_id}/reset

So provide:

[ Reset Progress ]

But **don't reset immediately.**

Show confirmation:

|                              |  |
|------------------------------|--|
| Reset Learning Progress?     |  |
|                              |  |
| This will reset your current |  |
| learning progress.           |  |
|                              |  |
| [Cancel] [Reset Progress]    |  |

Then:

Confirm



PUT /progress-tracking/{resume\_id}/reset



Progress becomes 0



Refresh UI



## 9 Loading state

When opening the page:

 Learning Progress

Loading your progress...

Use skeletons rather than an empty page.

---

## 10 No progress yet

If the user has not started learning:

 Learning Progress

0%

You haven't started your learning journey yet.

Start learning your recommended skills  
and update your progress as you go.

[View Learning Resources]

This creates a useful connection:

No Progress



Learning Resources

---

## Error state

If the API fails:

 Unable to load your learning progress.

Something went wrong.

[Retry]

---

## After updating

After the user saves:

PUT /progress-tracking/{resume\_id}

show a small success notification:

 Learning progress updated successfully.

Then refresh the displayed values.



Dashboard



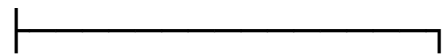
Career Coach



GET /career-chat/{resume\_id}



Loading



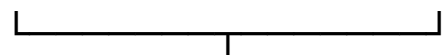
History exists

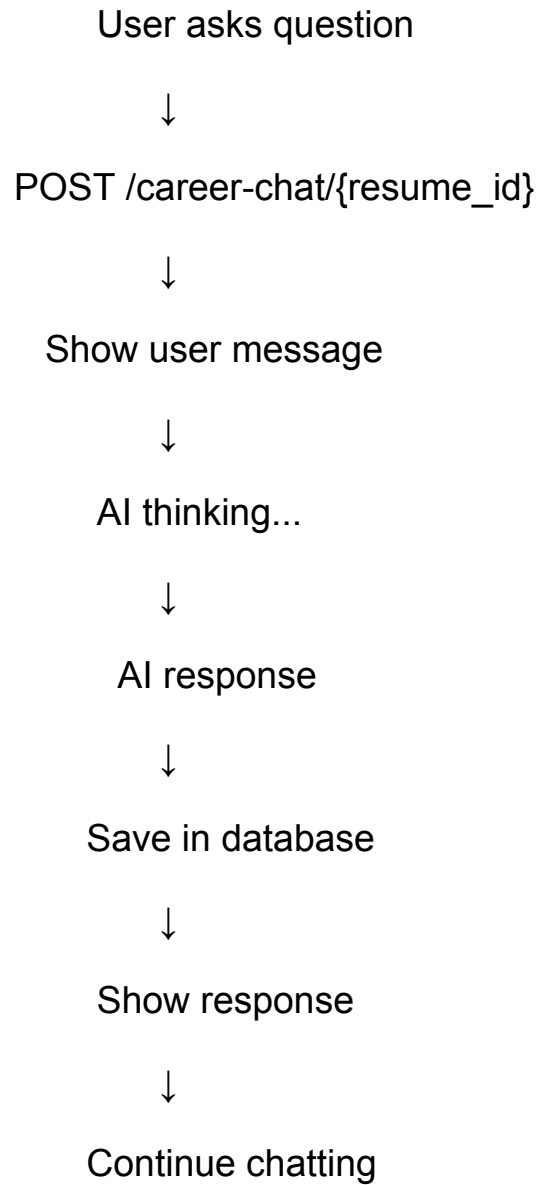
No history



Show messages

Welcome screen





## 2 Chat layout





AI Career Coach

[Clear Chat]



AI Coach

Hi! I've analyzed your resume.

Ask me anything about your career, skills,

learning roadmap, or interview preparation.

You

Should I learn Docker before

Kubernetes?



AI Coach

Yes. Docker should come first because...

Ask your career question... [Send]

### 3 Initial empty state

When the user opens Career Coach for the first time and has no chat history:



AI Career Coach

Hi! I'm your personal career coach.

I can help you with:

- Career roadmap
- Missing skills
- Learning priorities
- Interview preparation
- Projects
- Technologies to learn
- Career decisions

Ask me anything about your career.

Then input:

[ Type your question here... ] [➤]

## **4** Suggested questions

This makes the first interaction easier.

For example:

 Try asking

[ What should I learn next? ]

[ Should I learn Docker before Kubernetes? ]

[ How can I improve my missing skills? ]

[ What projects should I build? ]

## **7** AI response

Example:



AI Career Coach

Yes, you should learn Docker before Kubernetes.

Docker gives you the foundation of containerization,  
while Kubernetes focuses on managing containers at scale.

For your current roadmap, I recommend:

1. Docker fundamentals
2. Docker Compose
3. Containerize your FastAPI project
4. Basic CI/CD
5. Kubernetes fundamentals

## **8 Chat history**

When the user comes back to Career Coach:

GET /career-chat/{resume\_id}

Then display:

Previous conversation

You:

Should I learn Docker before Kubernetes?

AI:

Yes, you should learn Docker first...

You:

What should I build with Docker?

AI:

You should containerize your FastAPI project...

## 9 New message

User can continue:

[ What should I learn after Docker? ] [Send]

Then:

POST /career-chat/{resume\_id}

The new conversation appears at the bottom.

The chat should behave like a normal messaging application:

Old messages



New user message



AI response



Auto-scroll to latest message

---

## 10 Clear Conversation

You already have a clear/delete API.

So provide:

[ 🗑 Clear Conversation ]

But never delete immediately.

Show confirmation:

Clear Conversation?

This will permanently remove your career chat history.

[Cancel]

[Clear Chat]

After confirmation:

DELETE /career-chat/{resume\_id}



Database chat history deleted



Frontend becomes empty state

---

## Error state

If AI/API fails:

 Something went wrong.

I couldn't generate a response right now.

[Try Again]